```
`timescale 1ns / 1ps

module WB_tb;

    // Testbench signals
    reg clk;
    reg rst_n;
    reg i_RegSrc;
    reg [4:0] i_Rd;
    reg [1:0] i_ResultSrc;
    reg [31:0] i_Result;
    reg [31:0] i_Wb_Data;
    reg [31:0] i_Pc_4;

    wire [4:0] o_Rd;
    wire o_RegSrc;
    wire [31:0] o_Wb_data;

    // Instantiate the WB module
    WB uut (
        .clk(clk),
        .rst_n(rst_n),
        .i_RegSrc(i_RegSrc),
        .i_Rd(i_Rd),
        .i_ResultSrc(i_ResultSrc),
        .i_Result(i_Result),
        .i_Wb_Data(i_Wb_Data),
        .i_Pc_4(i_Pc_4),
        .o_Rd(o_Rd),
        .o_RegSrc(o_RegSrc),
        .o_Wb_data(o_Wb_data)
    );

    // Clock generation
    always begin
        #5 clk = ~clk; // 100MHz clock period
    end

    // Initial block to apply test vectors
    initial begin
        // Initialize signals
        clk = 0;
        rst_n = 0;
        i_RegSrc = 0;
        i_Rd = 5'b00000;
        i_ResultSrc = 2'b00;
```

```
        i_Result = 32'h00000000;
        i_Wb_Data = 32'h00000000;
        i_Pc_4 = 32'h00000004;

        // Apply reset
        #10 rst_n = 1;

        // Test case 1: Write ALU/AMO result (ResultSrc = 00)
        i_ResultSrc = 2'b00;
        i_Result = 32'h12345678;
        #10;
        $display("Test Case 1 - WB Data: %h", o_Wb_data);

        // Test case 2: Write memory result (ResultSrc = 01)
        i_ResultSrc = 2'b01;
        i_Wb_Data = 32'hAABBCCDD;
        #10;
        $display("Test Case 2 - WB Data: %h", o_Wb_data);

        // Test case 3: Write PC+4 result (ResultSrc = 10)
        i_ResultSrc = 2'b10;
        i_Pc_4 = 32'h10000004;
        #10;
        $display("Test Case 3 - WB Data: %h", o_Wb_data);

        // Test case 4: Check reset behavior
        rst_n = 0;
        #10;
        $display("Test Case 4 - WB Data (after reset): %h", o_Wb_data);

        // Finish simulation
        $finish;
    end

endmodule
```